

Технологии разработки программного обеспечения

Сборка проектов

Как запустить прогу?

Как запустить прогу?

- `g++ prog1.cpp prog2.cpp prog123.cpp`
- `./a.out (a.exe)`

Как запустить прогу?

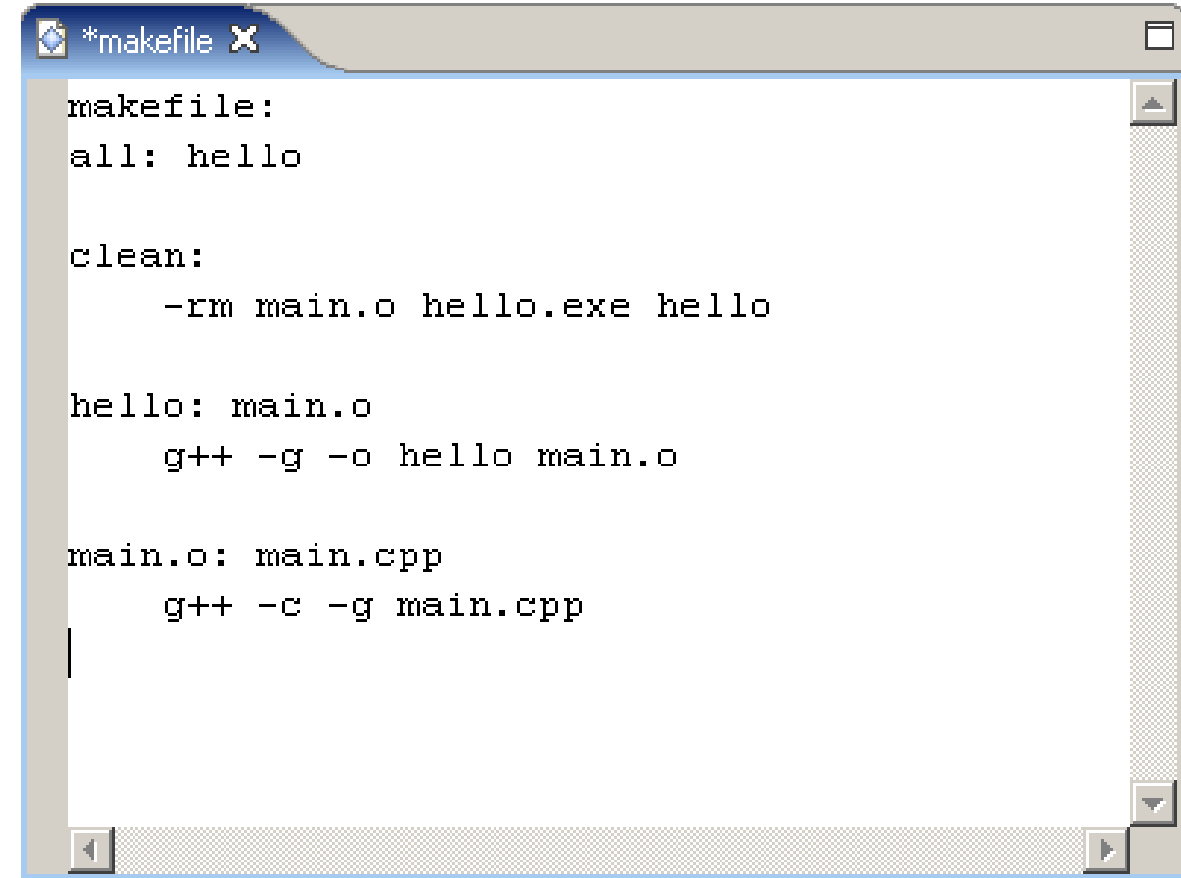
- `g++ prog1.cpp prog2.cpp prog123.cpp prog1.cpp prog2.cpp
prog123.cpp prog1.cpp prog2.cpp prog123.cpp prog1.cpp
prog2.cpp prog123.cpp prog1.cpp prog2.cpp prog123.cpp
prog1.cpp prog2.cpp prog123.cpp prog1.cpp prog2.cpp
prog123.cpp prog1.cpp prog2.cpp prog123.cpp prog1.cpp
prog2.cpp prog123.cpp prog1.cpp prog2.cpp prog123.cpp
prog1.cpp prog2.cpp prog123.cpp prog1.cpp prog2.cpp
prog123.cpp prog1.cpp prog2.cpp prog123.cpp prog1.cpp`
- `./a.out (a.exe)`

Как запустить прогу?

- `g++ *.cpp`
- `./a.out (a.exe)`

Автоматизация сборки

- **make** – инструмент для автоматизированной и параметризованной сборки проекта в единую программу
- **Makefile** – файл с инструкциями по сборке проекта



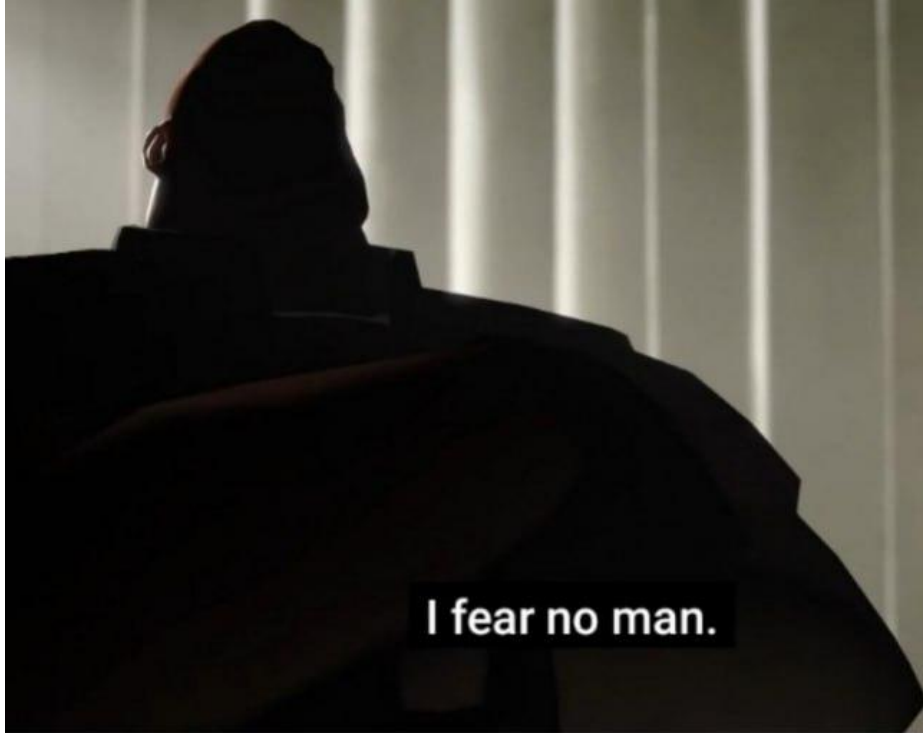
```
makefile:
all: hello

clean:
    -rm main.o hello.exe hello

hello: main.o
    g++ -g -o hello main.o

main.o: main.cpp
    g++ -c -g main.cpp
```

CMake



I fear no man.



But that thing...



it scares me.

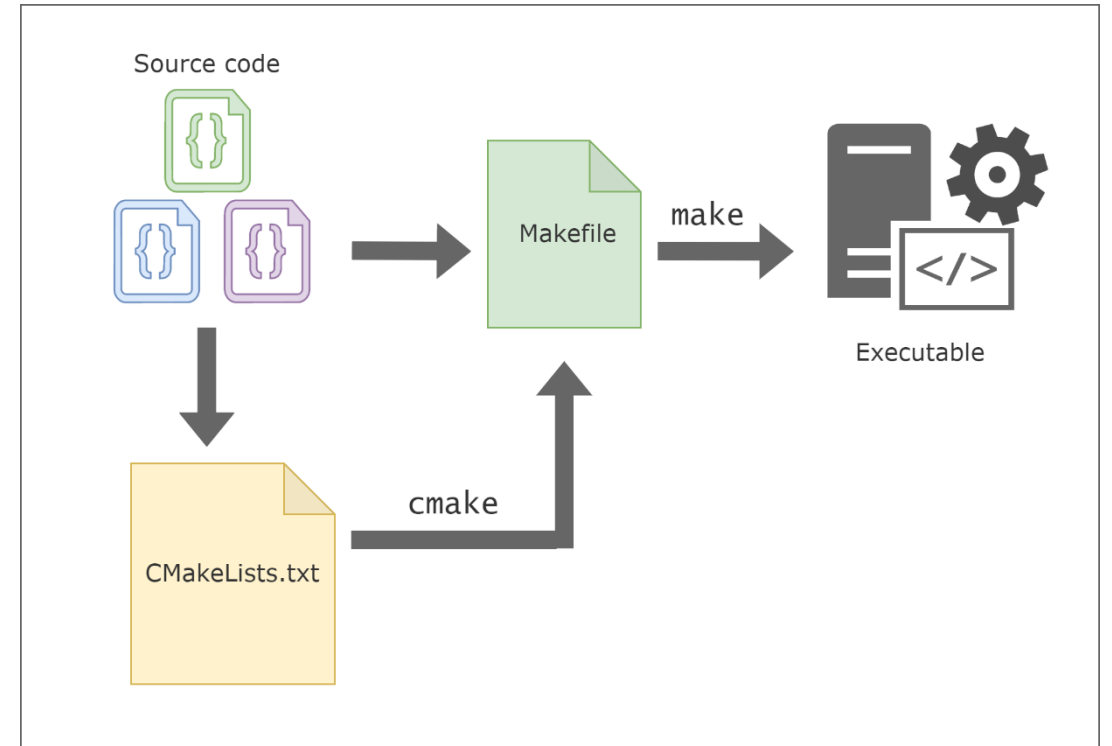
CMake – «стандарт» для сборки проектов

Плюсы:

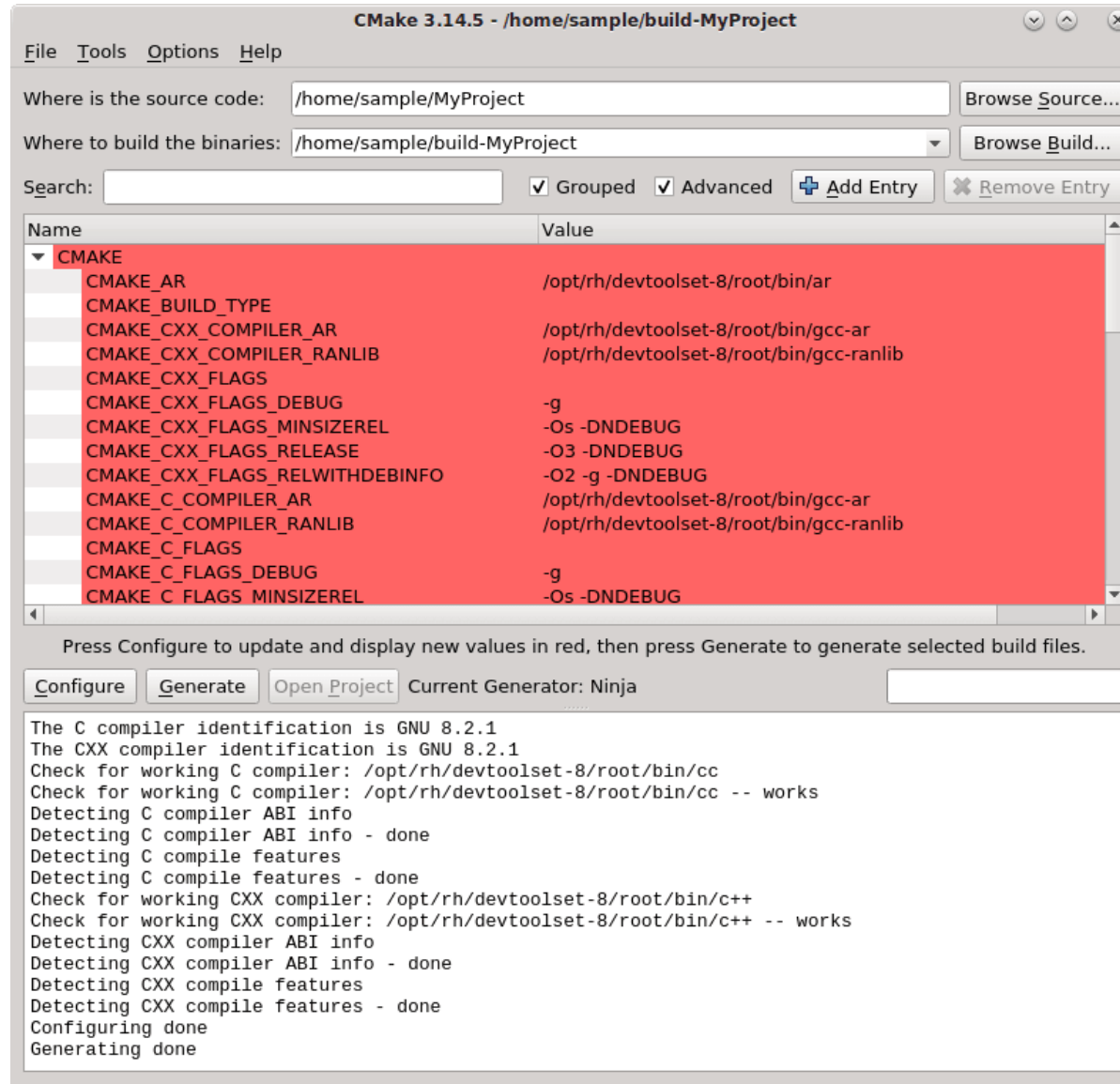
- Кроссплатформенность
- Легкое управление библиотеками
- «Человеческий» язык

Минусы:

- Сложность в обучении
- Неявное поведение



CMake GUI



Синтаксис

CMakeLists.txt

```
project(MyProjectName)

set(CMAKE_CXX_STANDARD 17)

add_executable(${PROJECT_NAME}
    main.cpp
    source1.cpp
    source2.cpp
)

target_link_libraries(${PROJECT_NAME}
    mylib1
)
```

```
cmake -S . -B build -G Ninja -
DCMAKE_BUILD_TYPE=Debug
```

```
cmake --build build
```

Виды конфигураций

- **Release:**
 - Максимальная скорость работы
 - Сокращение операций (g++ -O3)
- **Debug:**
 - Максимальная прозрачность (.pdb – Windows)
 - Сохранение операций в изначальном виде
- **MinRelSize** (Minimum Release Size) – для IoT, ...
- **RelWithDebInfo** (Release with Debug Info)



Тестирование



Тестирование

